

SecureRAG: Permission-Aware Multi-Tenant RAG

Technical Study Guide & Interview Preparation Manual

SecureRAG: Permission-Aware Multi-Tenant Document Intelligence Platform

Technical Study Guide & Interview Preparation Manual

SECTION 1: PROJECT OVERVIEW

What is SecureRAG?

SecureRAG is an enterprise-grade, production-oriented Retrieval-Augmented Generation (RAG) platform. It enables users from different organizations (tenants) and departments to upload PDF documents and ask questions in natural language, receiving grounded answers generated by a Large Language Model (Groq Llama / GPT-OSS) accompanied by exact page-level source citations.

What Problem Does It Solve?

Standard RAG systems ingest documents into a single global vector store and retrieve passages based purely on mathematical semantic similarity. In multi-tenant enterprise environments, this causes severe security vulnerabilities:

1. **Cross-Tenant Data Leakage:** Company A's users could accidentally retrieve confidential documents belonging to Company B.
2. **Privilege Escalation:** A junior Engineering employee could retrieve restricted executive payroll or HR grievance records.

SecureRAG solves this problem by embedding permission metadata (`tenant_id`, `allowed_roles`, `department`) directly onto document chunks and enforcing strict filtering in the backend **before** retrieved context is passed to the LLM.

Key Definitions

- **Multi-Tenancy:** An architecture where a single software instance serves multiple distinct customer organizations (tenants), ensuring complete data isolation.
- **Tenant Isolation:** A hard security boundary ensuring Tenant A can never view, search, or infer data belonging to Tenant B.
- **Role-Based Access Control (RBAC):** Restricting document access based on user roles (HR, ENGINEERING, FINANCE, ADMIN).
- **Permission-Aware RAG:** Retrieval where mathematical similarity search is constrained by real-time tenant and role authorization logic.

Complete Architectural Flow

```

[Streamlit UI Dashboard]
  ■ (HTTP POST /chat or /documents/upload)
  ▼
[FastAPI Backend Routes] (main.py, routes/chat.py, routes/documents.py)
  ■ (User Context: user_id, tenant_id, role)
  ▼
[Document Ingestion & Parsing] (PyPDFLoader in rag/ingestion.py)
  ■ (Extracts text & page metadata)
  ▼
[Text Chunking] (RecursiveCharacterTextSplitter in rag/chunking.py: 600 chars, 100 overlap)
  ■ (Attaches tenant_id, allowed_roles, page to every chunk)
  ▼
[Embedding Generation] (HuggingFace BAAI/bge-small-en-v1.5: 384d dense vectors)
  ■
  ▼
[FAISS Vector Store Persistence] (vectorstore/faiss_store.py: data/faiss_index)
  ■ (Similarity Search over-samples k*3 candidates)
  ▼
[Permission Filtering Engine] (services/permission_service.py)
  ■ (Drops cross-tenant & role-unauthorized chunks)
  ▼
[Authorized Context Assembly] (services/retrieval_service.py)
  ■ (Formats strict zero-hallucination prompt template)
  ▼
[Groq LLM Generation] (services/llm_service.py: openai/gpt-oss-120b)
  ■ (Grounded answer or standard fallback response)
  ▼
[Grounded Answer + Page-Level Source Citations] ■■■ Rendered on Streamlit UI

```

SECTION 2: RAG FUNDAMENTALS

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances Large Language Models by fetching relevant external facts from a private knowledge base and injecting them into the LLM prompt at query time.

Why RAG Instead of Direct Prompting or Fine-Tuning?

- Context Window Limits:** Passing hundreds of multi-page PDFs directly into an LLM prompt is cost-prohibitive, slow, and exceeds token limits.
- Dynamic Knowledge Updates:** Fine-tuning an LLM to learn new enterprise policy documents requires millions of parameters to be re-trained. With RAG, updating the knowledge base takes seconds (upload PDF -> index vectors).
- Hallucination Control:** Fine-tuned models memorize patterns statically and can hallucinate smoothly. RAG provides exact text passages for the LLM to cite.

Core RAG Concepts

- **Retrieval:** Vector similarity search locating the top- k most relevant document chunks.
- **Generation:** LLM processing retrieved chunks to write a fluent, human-readable response.
- **Grounded Generation:** Forcing the LLM to base its response strictly on retrieved facts.
- **Hallucination:** When an LLM invents plausible-sounding but false statements.

- **Why RAG Reduces Hallucinations:** RAG constrains the model's generation space to provided facts. However, RAG cannot guarantee zero hallucinations unless enforced with strict prompt templates and fallbacks (e.g. returning "I couldn't find this information in the authorized documents.").

SECTION 3: PDF INGESTION

How Documents Enter SecureRAG

1. User uploads a PDF via Streamlit sidebar, selecting target Department and Allowed Roles.
2. Streamlit sends a multipart/form-data request to FastAPI /documents/upload.
3. `load_pdf_document()` in `backend/rag/ingestion.py` uses `PyPDFLoader` to extract plain text page-by-page.

Text Chunking Parameters & Trade-offs

Raw page text is split using `RecursiveCharacterTextSplitter` in `backend/rag/chunking.py`:

- **chunk_size:** 600 characters (optimal balance between embedding specificity and context size).
- **chunk_overlap:** 100 characters (prevents loss of semantic context across chunk boundaries).
- **Separators:** `["\n\n", "\n", ". ", " ", ""]` (splits recursively along natural paragraph/sentence boundaries).

Small Chunks vs Large Chunks Trade-off:

- **Too Small (<200 chars):** Fragmented sentences lose context; vector embeddings become noisy.
- **Too Large (>2000 chars):** Dilutes specific facts; vector embeddings represent general topics rather than targeted answers.

SECTION 4: EMBEDDINGS

What is an Embedding?

An embedding is a numerical vector representation of text in a high-dimensional space where semantically similar phrases are located close to each other.

Model Used in SecureRAG: BAAI/bge-small-en-v1.5

- **Provider:** Hugging Face via `SentenceTransformers` / `HuggingFaceEmbeddings`.
- **Dimensions:** 384 floating-point numbers per vector.
- **Normalisation:** `encode_kwargs={'normalize_embeddings': True}` enables fast cosine distance computation.

Hugging Face vs Embedding Models vs Generative LLMs

- **Hugging Face:** An open-source model hub and library ecosystem (not an LLM itself).
- **Embedding Model (BAAI/bge-small-en-v1.5):** An encoder model that turns text into dense math vectors. It cannot converse or generate text.
- **Generative LLM (openai/gpt-oss-120b on Groq):** An autoregressive text generation model that predicts the next token to write answers.

SECTION 5: FAISS (FACEBOOK AI SIMILARITY SEARCH)

What is FAISS?

FAISS is an open-source, ultra-fast vector similarity search library developed by Meta. It stores dense vectors and performs nearest-neighbor search using L2/Cosine distance.

Implementation in SecureRAG (backend/vectorstore/faiss_store.py)

- **Docstore:** FAISS maintains an internal key-value mapping connecting each vector ID to its corresponding LangChain Document (text content + metadata dictionary).
- **Disk Persistence:** `faiss_store.save_local("data/faiss_index")` and `FAISS.load_local()` persist vector indices across server restarts.
- **Over-Sampling Strategy:** When querying, `PermissionAwareRetriever` queries FAISS with `fetch_k = top_k * 3` (e.g. 12 candidates) to ensure enough valid chunks remain after permission filtering.

SECTION 6: MULTI-TENANCY

Why Multi-Tenant Isolation Matters

In enterprise SaaS, multiple clients share the same cloud deployment. A failure in multi-tenancy results in catastrophic privacy breaches.

Demo User Accounts in SecureRAG

- rahul: `tenant_id = "company_a", role = "HR"`
- amit: `tenant_id = "company_a", role = "ENGINEERING"`
- admin_a: `tenant_id = "company_a", role = "ADMIN"`
- neha: `tenant_id = "company_b", role = "HR"`
- admin_b: `tenant_id = "company_b", role = "ADMIN"`

SECTION 7: ROLE-BASED ACCESS CONTROL (RBAC)

Metadata Structure Attached to Every Chunk

```
{
  "tenant_id": "company_a",
  "document_id": "doc_employee_policy",
  "document_name": "employee_policy.pdf",
  "department": "HR",
  "allowed_roles": ["HR", "ADMIN"],
  "page": 4,
  "chunk_id": "doc_employee_policy_p4_c0"
}
```

Permission Check Logic (backend/services/permission_service.py)

```
def is_chunk_authorized(self, chunk: Document, user: UserContext) -> bool:
    metadata = chunk.metadata or {}

    # 1. Tenant Check: Hard Isolation
    if metadata.get("tenant_id") != user.tenant_id:
        return False

    # 2. Role Check: RBAC
    user_role = user.role.upper()
    if user_role == "ADMIN":
        return True

    allowed_roles = [r.upper() for r in metadata.get("allowed_roles", [])]
    return user_role in allowed_roles
```

SECTION 8: PERMISSION-AWARE RETRIEVAL

Normal RAG vs SecureRAG Pipeline

```
Naive RAG:      Query [ ] [ ] [ ] Vector Search [ ] [ ] [ ] Top Chunks [ ] [ ] [ ] LLM (Security Leak Risk!)

SecureRAG:      Query [ ] [ ] [ ] Vector Search (Candidate Chunks)
                  [ ]
                  ▼
                PermissionService (Filter by tenant_id & allowed_roles)
                  [ ]
                  ▼
    Authorized Chunks ONLY [ ] [ ] [ ] Grounded Prompt [ ] [ ] [ ] LLM
```

Critical Security Rule

Unauthorized chunks are NEVER passed into the LLM prompt. Filtering happens strictly in the backend retrieval service. If no chunks pass permission validation, the system immediately returns the zero-hallucination fallback:

```
> "I couldn't find this information in the authorized documents."
```

SECTION 9: LLM INTEGRATION (GROQ API)

Primary & Fallback Models

- **Primary Model:** openai/gpt-oss-120b (Configured via GROQ_MODEL in .env).
- **Fallback Model:** qwen/qwen3.8-27b (Triggered automatically if primary model fails).
- **Inference Provider:** Groq LPU (Language Processing Unit) Cloud API for ultra-low latency inference.
- **Parameters:** temperature=0.1, max_tokens=1024 for deterministic factual answers.

SECTION 10: FASTAPI BACKEND

Implemented Endpoints

1. GET `/`: Health status and app name.
2. GET `/health`: Vector store and embedding model readiness status.
3. GET `/users`: List pre-configured demo user identity profiles.
4. GET `/users/{user_id}`: Fetch user identity by ID.
5. POST `/documents/upload`: Upload PDF, parse text, split into chunks, attach metadata, index into FAISS.
6. GET `/documents`: List indexed documents authorized for the requesting user.
7. POST `/chat`: Execute permission-aware RAG query and return answer + source citations.

SECTION 11: STREAMLIT FRONTEND

Key UI Features (`frontend/app.py`)

- **Sidebar Identity Selector**: Switch between Rahul, Amit, Neha, Admin A, Admin B in real time.
- **Document Uploader**: Upload PDF files with custom Department and Allowed Roles selection.
- **Interactive Chat Tab**: Submit queries, configure `top_k`, view grounded answers and formatted source badges.
- **Debug Chunk Inspector**: Expandable view showing exact retrieved chunks, similarity text, and security metadata.
- **Document Catalog Tab**: View table of all indexed documents accessible to the active tenant/role.

SECTION 12: SOURCE CITATIONS

Format & Extraction

Every generated answer is accompanied by structured citation objects:

```
[
  {
    "document_name": "employee_policy.pdf",
    "page": 4,
    "department": "HR"
  }
]
```

This enables users to inspect exact source pages supporting the answer, building trust and enabling rapid auditing.

SECTION 13: ERROR HANDLING

Covered Failure Scenarios

- **Missing GROQ_API_KEY:** Returns descriptive warning without crashing backend.
- **Groq API 404 / Model Error:** Triggers automatic fallback model attempt before returning 500 error.
- **Empty Query / File:** Pydantic and route validation return clear 400 Bad Request responses.
- **Cross-Tenant Access Attempt:** Silent backend drop of unauthorized chunks, returning standard fallback response.
- **Non-PDF Upload:** Upload route rejects unsupported extensions with 400 Bad Request.

SECTION 14: TESTING (PYTEST SUITE)

Automated Test Categories (10/10 Passed)

1. `test_root_endpoint`: Validates `/` status response.
2. `test_health_endpoint`: Validates `/health` backend readiness.
3. `test_role_based_access_control`: Validates HR vs ENGINEERING role checks.
4. `test_filter_chunks`: Validates chunk list filtering by user role.
5. `test_phase1_ingestion_and_vectorstore`: End-to-end PDF loading, chunking, and FAISS indexing test.
6. `test_context_formatting_and_citation_extraction`: Validates source header formatting.
7. `test_empty_context_fallback`: Validates instant fallback string return when context is empty.
8. `test_retrieval_service_with_empty_chunks`: Validates retrieval pipeline with 0 matching chunks.
9. `test_tenant_isolation_denies_cross_tenant_access`: Verifies Company A user cannot retrieve Company B chunk.
10. `test_tenant_isolation_allows_same_tenant_access`: Verifies Company B user retrieves Company B chunk.

SECTION 15: REAL MANUAL TEST SCENARIOS

Scenario 1: Authorized HR Access

- **User:** Rahul (company_a, HR)
- **Query:** "What is the annual leave policy?"
- **Outcome:** Successfully retrieves `employee_policy.pdf` (Page 4). Answer: 18 annual paid leave days.

Scenario 2: Cross-Tenant Denial

- **User:** Neha (company_b, HR)
- **Query:** "What is Company A's leave policy?"
- **Outcome:** Permission filter drops all Company A chunks. Result: "I couldn't find this information in the authorized documents."

Scenario 3: Role-Based Denial within Same Tenant

- **User:** Amit (company_a, ENGINEERING)

- **Query:** "What is the executive bonus structure?" (Restricted to HR/ADMIN)
- **Outcome:** Permission filter drops HR-restricted chunks. Result: "I couldn't find this information in the authorized documents."

SECTION 16: COMPLETE STEP-BY-STEP DATA FLOW

1. User **Amit** (company_a, ENGINEERING) submits question: `"What is the standard production deployment window?"`
2. Streamlit sends POST request to FastAPI /chat with payload `{user_id: "amit", tenant_id: "company_a", role: "ENGINEERING", question: "..."}.`
3. chat_endpoint passes user context to PermissionAwareRetriever.
4. Query string is converted into a 384d embedding using BAAI/bge-small-en-v1.5.
5. FAISS searches top 12 nearest neighbor vector chunks in data/faiss_index.
6. PermissionService checks each candidate chunk:
 - Chunk 1 (tenant_id="company_a", allowed_roles=["ENGINEERING", "ADMIN"]) -> **ALLOWED**.
 - Chunk 2 (tenant_id="company_b", allowed_roles=["ENGINEERING"]) -> **DENIED (Tenant mismatch)**.
 - Chunk 3 (tenant_id="company_a", allowed_roles=["HR"]) -> **DENIED (Role mismatch)**.
7. Top 4 authorized chunks are formatted into USER_PROMPT_TEMPLATE.
8. Formatted prompt sent to Groq API (openai/gpt-oss-120b).
9. Groq returns grounded answer: `"Production deployments occur on Tuesdays between 2 AM and 4 AM EST."`
10. AuditService appends event to data/audit_logs.jsonl.
11. FastAPI returns JSON payload containing answer, citations (deployment_guide.pdf, Page 12), and chunk previews to Streamlit.

SECTION 17: IMPORTANT INTERVIEW CONCEPTS

- **RAG:** Architecture combining document vector retrieval with LLM generation for accurate factual QA.
- **Embeddings:** Numerical vector representations of text preserving semantic proximity.
- **FAISS:** Library for high-efficiency dense vector similarity search and index persistence.
- **Top-K Retrieval:** Fetching the k closest vector matches to a query embedding.
- **Chunk Overlap:** Duplicating trailing characters across adjacent text chunks to preserve edge context.
- **Grounded Generation:** Restricting LLM answer generation strictly to provided context facts.
- **Tenant Isolation:** Hard security boundary preventing multi-tenant cross-contamination.
- **RBAC:** Access control based on user role assignments (HR, ENGINEERING, ADMIN).
- **Metadata Filtering:** Evaluating chunk metadata tags before releasing text to LLMs.
- **Audit Trail:** Append-only log file recording security and operational events for compliance.

SECTION 18: COMPREHENSIVE INTERVIEW QUESTIONS & ANSWERS

Q1: Explain your SecureRAG project in 2 minutes.

- **Short Answer:** SecureRAG is a permission-aware multi-tenant RAG platform built with FastAPI, Streamlit, LangChain, FAISS, and Groq Llama/GPT-OSS LLMs.
- **Detailed Explanation:** It solves data leakage in multi-tenant RAG applications by tagging document chunks with `tenant_id` and `allowed_roles` metadata during PDF ingestion. During search, a permission filtering layer evaluates candidate vector matches against the requesting user's identity before assembling the LLM prompt.
- **SecureRAG Example:** If Rahul (`company_a`, HR) queries the system, FAISS retrieves candidates, but any chunks belonging to `company_b` or restricted to `ENGINEERING` are filtered out before calling Groq.
- **Interview Tip:** Emphasize that security happens **before** the LLM prompt construction, not in client code.

Q2: Why use RAG instead of fine-tuning the LLM?

- **Short Answer:** RAG is cheaper, instant to update, and provides exact source citations while avoiding hallucinated memorization.
- **Detailed Explanation:** Fine-tuning alters model weights to learn domain style or static facts, which requires GPU clusters and re-training whenever policies change. RAG keeps the LLM frozen and injects live documents at query time.
- **SecureRAG Example:** When a company uploads a new `employee_policy.pdf`, SecureRAG indexes it into FAISS in under 2 seconds without retraining any models.
- **Interview Tip:** Highlight the difference between knowledge injection (RAG) and style adaptation (Fine-tuning).

Q3: How does your permission-aware retrieval prevent cross-tenant data leaks?

- **Short Answer:** By enforcing strict `tenant_id` and role metadata matching in the backend before prompt construction.
- **Detailed Explanation:** In `backend/services/permission_service.py`, `is_chunk_authorized()` evaluates every candidate vector returned by FAISS. If `chunk.metadata.tenant_id != user.tenant_id`, the chunk is dropped immediately.
- **SecureRAG Example:** User Neha from Company B asks about Company A's leave policy. FAISS finds similarity matches, but permission filtering drops 100% of Company A chunks, returning "I couldn't find this information in the authorized documents."
- **Interview Tip:** Stress that unauthorized chunks never enter the LLM's context window.

Q4: Why did you choose BAAI/bge-small-en-v1.5 for embeddings?

- **Short Answer:** It is an efficient, top-performing open-source retrieval embedding model producing 384-dimensional dense vectors.
- **Detailed Explanation:** BGE-small balances high retrieval accuracy on benchmarks with fast CPU vector computation, making it lightweight and ideal for local vector persistence with FAISS.
- **SecureRAG Example:** Configured via `HuggingFaceEmbeddings` in `backend/services/embedding_service.py`.
- **Interview Tip:** Know vector dimensionality (384 dimensions) for your embedding model.

Q5: What happens if no relevant or authorized documents are found?

- **Short Answer:** The system triggers a strict zero-hallucination fallback response.
- **Detailed Explanation:** If permission filtering removes all candidates or retrieved chunks have zero context, `LLMService` returns "I couldn't find this information in the authorized documents." without making hallucinated guesses.
- **SecureRAG Example:** Tested in `test_empty_context_fallback` in `tests/test_phase2.py`.

- **Interview Tip:** Explain how prompt engineering (temperature=0.1 + strict system instructions) prevents the LLM from falling back to general pre-training knowledge.

SECTION 19: PRODUCTION IMPROVEMENTS (FUTURE SCOPE)

| Feature Area | Current PoC Implementation | Production Architecture Version |

| :--- | :--- | :--- |

| **Authentication** | Demo User Context Selector | OAuth2 / OpenID Connect + JWT Tokens |

| **Vector Storage** | FAISS local file storage | Managed Qdrant / Pinecone / PostgreSQL pgvector |

| **Document Storage** | Local disk (data/uploads) | AWS S3 / Azure Blob Storage |

| **Reranking** | Bi-Encoder vector search only | Cross-Encoder Reranker (bge-reranker-large) |

| **Audit Logs** | Local JSONL append file | Distributed Elasticsearch / AWS CloudWatch |

| **Rate Limiting** | None | Redis Token Bucket Rate Limiter |

SECTION 20: FRESHER INTERVIEW PREPARATION CHECKLIST

MUST KNOW (Core Mastery Required)

- [x] What RAG is and how context injection works.
- [x] Embeddings vs Generative LLMs.
- [x] Vector similarity search concept (Cosine / L2 distance).
- [x] Chunking strategy (size, overlap, separators).
- [x] Multi-tenancy isolation and RBAC metadata tagging.
- [x] Why security filtering must happen before LLM prompt assembly.
- [x] FastAPI route handling, Pydantic schemas, and CORS.

GOOD TO KNOW (Distinguishes Top Candidates)

- [] Cross-Encoder reranking concepts.
- [] PostgreSQL pgvector indexing (HNSW vs IVFFlat).
- [] JWT authentication headers (Bearer token).
- [] RAG evaluation frameworks (Ragas / TruLens).

NOT NECESSARY FOR THIS PROJECT (Avoid Over-Engineering)

- LangGraph / Multi-Agent Frameworks (adds unnecessary state complexity for straightforward QA).
- Kubernetes / Kafka / Microservices (adds DevOps overhead to a focused RAG project).

SECTION 21: ONE-PAGE REVISION CHEAT SHEET

PROJECT: SecureRAG (Permission-Aware Multi-Tenant Document Intelligence Platform)

STACK: Python 3.14 | FastAPI | Streamlit | LangChain | FAISS | Hugging Face (bge-small 384d) | Groq (gpt-oss-120b) | Pydantic

CORE ARCHITECTURE:

PDF Upload █████ PyPDF Page Parsing █████ Recursive Chunking (600/100) █████ BGE Embeddings █████ FAISS Index (disk)

[illegible]

```

PermissionService Filter (tenant_id == user.tenant_id AND user.role in allowed_roles)

```

Authorized Chunks ONLY ■■■ Grounded Prompt (temp=0.1) ■■■ Groq LLM ■■■ Answer + Citations

GOLDEN RULES:

1. Unauthorized chunks NEVER enter LLM context window.
2. Missing context fallback: "I couldn't find this information in the authorized documents."
3. Every answer includes Page-Level Source Citations (Document Name, Page #, Department).
4. Tests: 10/10 automated Pytest suite passed (Health, Ingestion, RBAC, Tenant Isolation).